ESILV

Maîtriser le phénomène de Runge en interpolation polynomiale

Auteur : GOUDON Octave
Formation : 3ème année 2025/26

Année universitaire 2025–2026

Abstract

This report presents a study on the Runge phenomenon carried out as part of . The objective is to illustrate this phenomenon, to present the main theoretical and methodological elements aimed at limiting it, and to discuss the results obtained in a clear and rigorous manner.

Keywords: Polynomial interpolation, the Runge phenomenon, interpolation at Chebyshev nodes.

Résumé

Ce rapport présente une étude sur le phénomène de Runge réalisée dans le cadre de . L'objectif est d'illuster ce phénomène, de présenter les principaux éléments théoriques et méthodologiques afin de limiter ce phénomène, puis de discuter les résultats obtenus de manière claire et rigoureuse.

Mots-clés : interpolation polynomiale, phénomène de Runge, interpolation aux points de Tchebychev.

Table des matières

1 Introduction **3**

1.1 Contexte et état de l'art 3

1.2 Objectif du rapport 3

1.3 Organisation du rapport 3

2 Interpolation polynomiale **4**

2.1 Base de Lagrange 4

2.2 Algorithme de Newton 4

3 Points équidistants **5**

3.1 Présentation des points équidistants 5

3.2 Introduction des fonctions 5

3.3 Observations 6

3.4 Erreur 6

3.5 Vers une amelioration du modèle 7

4 Points de tchebychev **9**

4.1 Présentation des points de Tchebychev 9

4.2 Introduction des fonctions 9

4.3 Observations 9

4.4 Erreur 10

5 Conclusion **10**

A Annexe A : Démonstrations **11**

A.1 Erreur dans le cas des points équidistants 11

A.2 Erreur dans le cas des points de Tchebychev 11

B Annexe B : Programmes Matlab **12**

1 Introduction

Le phénomène de Runge correspond à une difficulté classique de l'interpolation polynomiale lorsque l'on utilise des points équidistants pour construire un polynôme de degré élevé. Dans ce contexte, au lieu d'obtenir une approximation de plus en plus précise de la fonction, le polynôme interpolateur développe des oscillations importantes, particulièrement visibles aux extrémités de l'intervalle. Ce comportement met en évidence l'instabilité inhérente aux polynômes de haut degré associés à une répartition uniforme des points, qui amplifie les erreurs et rend l'approximation globalement peu fiable. Le phénomène de Runge souligne ainsi les limites de l'interpolation polynomiale naïve et montre que la qualité de l'approximation dépend fortement du choix des points d'interpolation, ce qui justifie l'utilisation de distributions plus adaptées, comme les nœuds de Tchebychev, pour garantir une meilleure stabilité numérique.

1.1 Contexte et état de l'art

Soit $f \in \mathcal{C}^p([a,b],\mathbb{R})$ avec $p \geq 1$. On se donne des points distincts $x_0, x_1, \ldots, x_n \in [a,b]$ et l'on souhaite approcher certaines valeurs de f par un polynôme P tel que

$$\begin{cases} P(x_0) = f(x_0), \\ P(x_1) = f(x_1), \\ \quad \vdots \\ P(x_n) = f(x_n). \end{cases}$$

Un tel polynôme est appelé *polynôme d'interpolation* de f aux points $x_0, \ldots, x_n$.

1.2 Objectif du rapport

Dans ce rapport, nous nous intéressons à l'étude de l'interpolation polynomiale, et plus particulièrement à la construction du polynôme interpolateur de Lagrange ainsi qu'aux limites de cette méthode mises en évidence par le phénomène de Runge. L'objectif est d'analyser le comportement de l'interpolation en fonction du choix des points, de comprendre les mécanismes conduisant à l'apparition d'oscillations indésirables, et d'examiner des solutions permettant d'améliorer la stabilité de l'approximation, notamment l'utilisation des points de Tchebychev.

1.3 Organisation du rapport

Le rapport est structuré en trois grandes étapes.

Tout d'abord, la Section 2 pose le cadre théorique de l'interpolation polynomiale en détaillant la construction des polynômes dans la base de Lagrange et l'implémentation algorithmique via la méthode de Newton.

Ensuite, la Section 3 est consacrée à l'étude des points équidistants. Nous y analysons les performances de l'approximation sur différentes fonctions afin de mettre en évidence l'émergence du phénomène de Runge et d'explorer des pistes d'amélioration élémentaires (parité du nombre de points, restriction d'intervalle).

Enfin, la Section 4 présente l'alternative des points de Tchebychev. Cette partie expose les résultats obtenus avec une distribution non uniforme des nœuds et démontre l'efficacité de cette approche pour stabiliser l'interpolation et minimiser l'erreur globale.

2 Interpolation polynomiale

Cette section constitue le cœur du rapport. Elle présente de manière progressive et pédagogique la problématique étudiée, les outils mobilisés, les hypothèses formulées et la méthode utilisée.

2.1 Base de Lagrange

Pour construire le polynôme d'interpolation de Lagrange associé aux points $x_0, x_1, \ldots, x_n$, on introduit les polynômes de base définis par

$$L_i(x) = \prod_{\substack{0 \le j \le n \\ j \neq i}} \frac{x - x_j}{x_i - x_j}, \qquad i = 0, 1, \ldots, n.$$

Ces polynômes vérifient la propriété fondamentale

$$L_i(x_j) = \begin{cases} 1 & \text{si } i = j, \\ 0 & \text{si } i \neq j, \end{cases}$$

ce qui garantit que le polynôme interpolateur prend exactement les valeurs de f aux points imposés.

Le polynôme d'interpolation de Lagrange s'écrit alors

$$P(x) = \sum_{i=0}^{n} f(x_i)\, L_i(x),$$

ce qui permet d'estimer la valeur de f en tout point $x \in [a, b]$ à partir des valeurs connues $f(x_0), \ldots, f(x_n)$.

2.2 Algorithme de Newton

La méthode de Lagrange présente un inconvénient majeur : l'ajout d'un nouveau point impose de recalculer l'ensemble du polynôme. Pour pallier cette difficulté, on utilise la forme de Newton, qui permet une mise à jour incrémentale du polynôme interpolateur.

Les différences divisées sont organisées dans le tableau suivant :

x_i	$f[x_i]$	$f[x_i, x_{i+1}]$	$f[x_i, x_{i+1}, x_{i+2}]$	$\cdots$
x_0	$f(x_0)$	$f[x_0, x_1]$	$f[x_0, x_1, x_2]$	$\cdots$
x_1	$f(x_1)$	$f[x_1, x_2]$	$f[x_1, x_2, x_3]$	$\cdots$
x_2	$f(x_2)$	$f[x_2, x_3]$	$\vdots$	
$\vdots$	$\vdots$	$\vdots$		
x_n	$f(x_n)$			

Le polynôme d'interpolation de Newton s'écrit

$$P(x) = \sum_{k=0}^{n} a_k \prod_{j=0}^{k-1} (x - x_j),$$

où

$$a_0 = f[x_0], \quad a_1 = f[x_0, x_1], \quad a_2 = f[x_0, x_1, x_2], \ \ldots, \ a_n = f[x_0, x_1, \ldots, x_n].$$

Par convention, le produit pour $k = 0$ est vide et vaut 1, ce qui donne $P(x_0) = a_0$.

3 Points équidistants

Cette section présente les principaux résultats obtenus par interpolation polynomiale dans le cas de points équidistants.

3.1 Présentation des points équidistants

Dans cette étude, nous fixons l'intervalle $[a, b]$ avec $a = -5$ et $b = 5$. Pour un entier $n > 0$, nous considérons la subdivision régulière définie par

$$x_i = a + \frac{b-a}{n}, \qquad 0 \leq i \leq n,$$

ce qui fournit un ensemble de $n + 1$ points équidistants sur l'intervalle $[-5, 5]$.

3.2 Introduction des fonctions

Nous considérons deux fonctions régulières définies sur l'intervalle $[-5, 5]$:

$$f_1(x) = \cos(x), \qquad f_2(x) = \frac{1}{1 + x^2}.$$

3.3 Observations

À l'aide du programme MATLAB fourni en annexe, nous traçons les fonctions ainsi que leurs polynômes interpolateurs construits à partir des points équidistants définis précédemment.

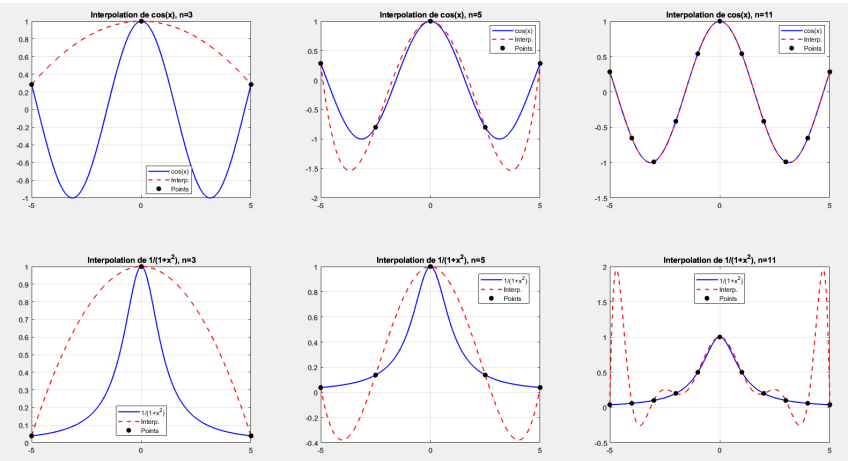

FIGURE 1 : Interpolation polynomiale dans le cas de points équidistants pour différentes valeurs de n.

Pour $f_1(x) = \cos(x)$, l'interpolation polynomiale suit correctement la fonction, même pour des degrés élevés. Cela montre que, dans ce cas, l'interpolation constitue une bonne méthode d'approximation.

En revanche, pour $f_2(x) = \frac{1}{1+x^2}$, on observe des oscillations importantes aux extrémités de l'intervalle lorsque n augmente : c'est le *phénomène de Runge*.

3.4 Erreur

Afin de confirmer ces observations, on étudie l'erreur d'interpolation. L'erreur est donnée par

$$|f(x) - P_n(x)| = \frac{f^{(n+1)}(\xi_x)}{(n+1)!} \prod_{i=0}^{n} (x - x_i),$$

où $\xi_x \in (a, b)$ dépend de x.

Dans le cas de points équidistants, on montre en annexe que

$$\|f - P_n\|_\infty \leq \frac{\|f^{(n+1)}\|_\infty}{n!} h^{n+1} \frac{1}{4(n+1)},$$

où $h = \frac{b-a}{n}$ est le pas de la subdivision.

Pour la fonction de Runge $f(x) = \frac{1}{1+x^2}$, les dérivées successives $f^{(n+1)}$ croissent extrêmement vite avec n. Pour des points équidistants, le produit $q(x) = \prod_{i=0}^{n}(x - x_i)$, appelé polynôme nodal, prend des valeurs très importantes près des bords de l'intervalle. À mesure que n augmente, les

oscillations près de -5 et 5 deviennent incontrôlables. L'erreur maximale tend vers l'infini :

$$\lim_{n\to\infty}\left(\max_{x\in[-5,5]}|f(x)-P_n(x)|\right)=\infty$$

3.5 Vers une amelioration du modèle

Choix du nombre de points

Il est préférable, lorsque l'on travaille avec des fonctions centrées sur l'intervalle étudié, d'utiliser un nombre impair de points d'interpolation. Cela permet ici de disposer d'un point central exactement situé au milieu de l'intervalle, ce qui force le modèle à suivre la courbure de la fonction en considérant son sommet.

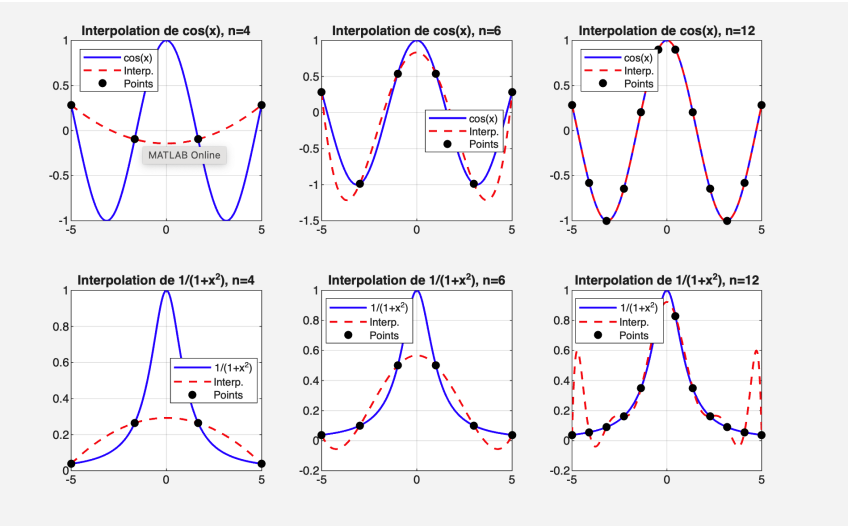

FIGURE 2 : Interpolation polynomiale dans le cas de points équidistants pour différentes valeurs paires de n.

Sur la figure 1, qui présente les fonctions f1 et f2 étudiées avec un nombre de points d'interpolation n = 3, 5 et 11, on observe une approximation globalement satisfaisante, en particulier grâce à l'utilisation d'un nombre impair de points permettant une meilleure symétrie autour du centre de l'intervalle. En revanche, la figure 2, qui illustre ces mêmes fonctions f1 et f2 pour des nombres de points d'interpolation n = 4, 6 et 11, met en évidence des erreurs bien plus importantes. Cette différence s'explique notamment par l'absence de point central lorsque le nombre de points est pair, ce qui dégrade la qualité de l'interpolation et accentue les écarts entre la fonction réelle et son approximation.

Choix de l'intervalle considéré

En décomposant l'intervalle étudié $[-5,5]$ en plusieurs sous-intervalles, à savoir $[-5,-1.5]$ (figure 3), $[-1.5,1.5]$ (figure 4) et $[1.5,5]$, on obtient de meilleures approximations pour la fonction f_2. Cette amélioration s'explique par le fait que, sur des intervalles plus restreints, la fonction est mieux approchée par un polynôme d'interpolation, les variations étant moins importantes et les phénomènes d'oscillation réduits. Ainsi, la décomposition de l'intervalle permet de limiter les

erreurs d'interpolation et d'obtenir une approximation plus fidèle du comportement de f_2 sur chaque portion considérée.

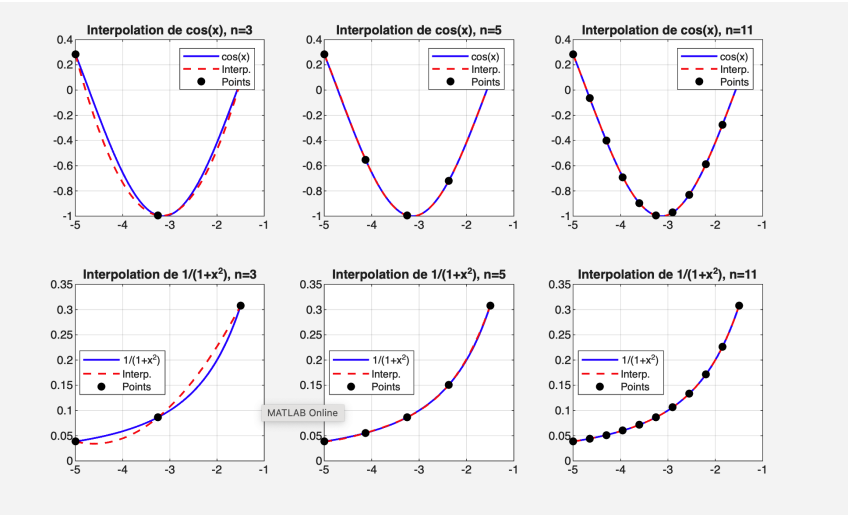

FIGURE 3 : Interpolation polynomiale dans le cas de points équidistants pour différentes valeurs de n sur le sous-intervalle [-5, -1.5].

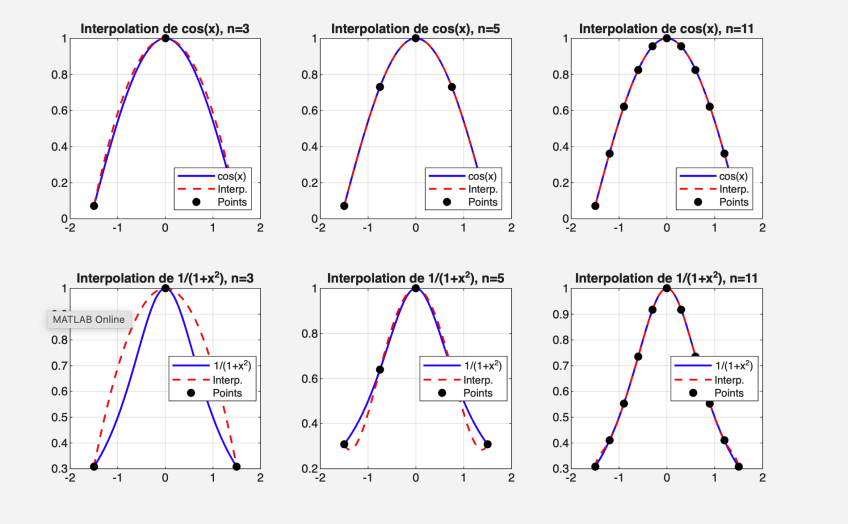

FIGURE 4 : Interpolation polynomiale dans le cas de points équidistants pour différentes valeurs de n sur le sous-intervalle [-1.5, 1.5].

Vers une amelioration du modèle

On a pu constater que plusieurs facteurs influencent fortement la qualité d'une interpolation polynomiale : le nombre total de points, la parité de ce nombre, ainsi que l'intervalle étudié. Autrement dit, l'emplacement des points d'interpolation joue un rôle crucial dans la précision de l'approximation. Pour limiter les erreurs, en particulier les oscillations aux extrémités, il est donc essentiel de choisir judicieusement la répartition des points. C'est pourquoi, dans la suite, nous allons nous intéresser aux points de Tchebychev, qui permettent de mieux répartir les nœuds et d'optimiser l'approximation polynomiale sur l'intervalle étudié.

4 Points de tchebychev

Cette section présente les princpaux résultats obtenus par interpolation polynomiale dans le cas de points de Tchebychev.

4.1 Présentation des points de Tchebychev

On appelle *points d'interpolation de Tchebychev* sur l'intervalle $[a, b]$(avec a=-5, b=5) les points x_i définis par

$$x_i = \frac{a+b}{2} + \frac{b-a}{2}\cos\left(\frac{2i+1}{2(n+1)}\pi\right), \qquad 0 \leq i \leq n.$$

4.2 Introduction des fonctions

Dans cette partie, on conservera les fonctions définis précedemment.

4.3 Observations

En modifiant les lignes permettant de calculer les points, on obtient les tracés suivants.

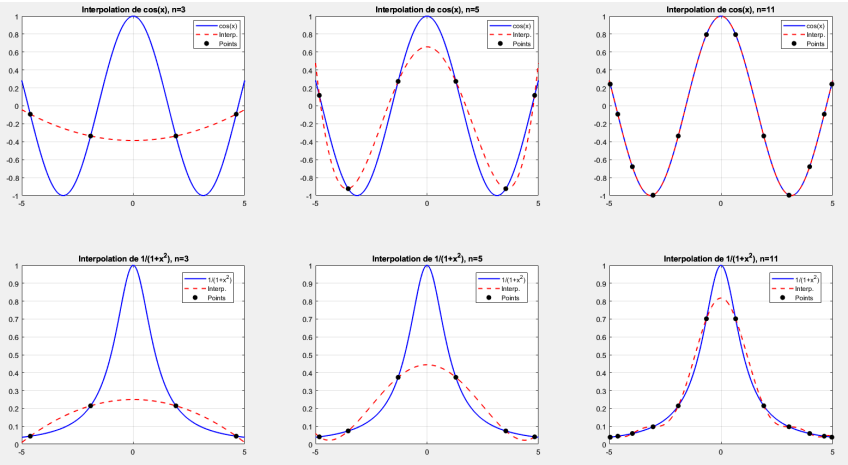

FIGURE 5 : Interpolation polynomiale dans le cas de points de TCHEBYCHEV pour différentes valeurs de n.

On constate maintant dans le cas de f1 que cela fonctionne toujours. Mais désormais, dans le cas de f2, l'interpolation semble suivre la fonction, minorant ainsi l'erreur, de manière plus efficace qu'avec la méthode des points équidistants.

4.4 Erreur

Afin de confirmer ces observations, on étudie l'erreur d'interpolation. L'erreur est donnée par

$$|f(x) - P_n(x)| = \frac{f^{(n+1)}(\xi_x)}{(n+1)!} \prod_{i=0}^{n} (x - x_i),$$

où $\xi_x \in (a, b)$ dépend de x.

Dans le cas des points de Tchebychev, on montre en annexe que :

$$E_n(f) \leq \frac{\|f^{(n+1)}\|_\infty}{(n+1)!} \frac{(b-a)^{n+1}}{2^{2n+1}}$$

où $h = \frac{b-a}{n}$ est le pas de la subdivision.

Stratégie de réduction d'erreur :

En resserrant les points aux extrémités, on réduit drastiquement la valeur du polynôme nodal $q(x)$ là où il est habituellement le plus grand dans le cas équidistant.

Propriété d'optimalité :

Les points de Tchebychev minimisent la valeur maximale du polynôme nodal sur l'intervalle :

$$\min_{\{x_i\}} \max_{x \in [a,b]} |\prod_{i=0}^{n} (x - x_i)|$$

C'est la répartition "optimale" qui minimise la borne supérieure de l'erreur d'interpolation.

Résultat :

Pour la fonction de Runge, cette distribution compense la croissance des dérivées. L'erreur diminue lorsque n augmente, garantissant la convergence uniforme de l'interpolant vers la fonction.

5 Conclusion

Les résultats obtenus montrent que si l'interpolation par l'algorithme de Newton sur des points équidistants est simple à mettre en œuvre, elle reste vulnérable aux fortes oscillations de bord (phénomène de Runge) dès que le degré du polynôme augmente. La substitution de ces points par les nœuds de Tchebychev s'est avérée concluante : en concentrant davantage d'informations aux extrémités de l'intervalle, cette méthode minimise l'erreur et garantit une bien meilleure fidélité à la fonction d'origine.

A Annexe A : Démonstrations

A.1 Erreur dans le cas des points équidistants

La formule générale de l'erreur : Pour toute interpolation polynomiale de degré n sur un intervalle [a,b], l'erreur en un point x est donnée par :

$$E_n(f) = f(x) - P_n(x) = \frac{f^{(n+1)}(\xi)}{(n+1)!} \prod_{i=0}^{n} (x - x_i)$$

Par consequent,

$$E_n(f) \leq \frac{\|f^{(n+1)}\|_\infty}{(n+1)!} \sup_{x \in [a,b]} \left| \prod_{i=0}^{n} (x - x_i) \right|$$

Or, dans le cas des points équidistants, on a :

$$\sup_{x \in [a,b]} \left| \prod_{i=0}^{n} (x - x_i) \right| \leq \frac{n!}{4} h^{n+1} \quad \text{avec} \quad h = \frac{b-a}{n}$$

Donc

$$E_n(f) \leq \frac{\|f^{(n+1)}\|_\infty}{4(n+1)} h^{n+1} \quad \text{avec} \quad h = \frac{b-a}{n}$$

A.2 Erreur dans le cas des points de Tchebychev

La formule générale de l'erreur :

Pour toute interpolation polynomiale de degré n sur un intervalle [a,b], l'erreur en un point x est donnée par :

$$E_n(f) = f(x) - P_n(x) = \frac{f^{(n+1)}(\xi)}{(n+1)!} \prod_{i=0}^{n} (x - x_i)$$

Par consequent,

$$E_n(f) \leq \frac{\|f^{(n+1)}\|_\infty}{(n+1)!} \sup_{x \in [a,b]} \left| \prod_{i=0}^{n} (x - x_i) \right|$$

Or, dans le cas des points de Tchebychev, on a :

Théorème :

$$\min_{(x_0,\dots,x_n)\in[a,b]^{n+1}} \sup_{x\in[a,b]} \left|\prod_{i=0}^{n}(x-x_i)\right| = \sup_{x\in[a,b]} \left|\prod_{i=0}^{n}(x-x_i)\right| = \frac{(b-a)^{n+1}}{2^{2n+1}}$$

Ainsi,

$$E_n(f) \leq \frac{\|f^{(n+1)}\|_\infty}{(n+1)!}\frac{(b-a)^{n+1}}{2^{2n+1}}$$

B Annexe B : Programmes Matlab

Programme MATLAB de tracé des courbes de la fonction interpolée et du polynôme interpolateur en utilisant l'algorithme de Newton avec la méthode des points équidistants.

```
% Interpolation par la méthode des différences divisées de Newton
clc;
clear;
close all;

% Définition des fonctions f1 et f2
f1 = @(x) cos(x);
f2 = @(x) 1 ./ (1 + x.^2);

% Bornes de l'intervalle
a = -5;
b = 5;

% Choix du nombre de points pour l'interpolation
n_values = [3, 5, 11];

% Evaluation et tracé pour chaque valeur de n
x_fine = linspace(a, b, 1000);
figure;

for idx = 1:length(n_values)
    n = n_values(idx);
    x_n = linspace(a, b, n); % Points d'interpolation
    y_f1 = f1(x_n);
    y_f2 = f2(x_n);

    % Calcul des coefficients de Newton
    coeffs_f1 = newton_coeffs(x_n, y_f1);
    coeffs_f2 = newton_coeffs(x_n, y_f2);

    % Evaluation des polynomes interpolants
    y_interp_f1 = newton_eval(coeffs_f1, x_n, x_fine);
```

```
    y_interp_f2 = newton_eval(coeffs_f2, x_n, x_fine);

    % Tracé des résultats
    subplot(2, length(n_values), idx);
    plot(x_fine, f1(x_fine), 'b-', 'LineWidth', 1.5); hold on;
    plot(x_fine, y_interp_f1, 'r--', 'LineWidth', 1.5);
    plot(x_n, y_f1, 'ko', 'MarkerFaceColor', 'k');
    title(['cos(x), n=' num2str(n)]);
    grid on;

    subplot(2, length(n_values), idx + length(n_values));
    plot(x_fine, f2(x_fine), 'b-', 'LineWidth', 1.5); hold on;
    plot(x_fine, y_interp_f2, 'r--', 'LineWidth', 1.5);
    plot(x_n, y_f2, 'ko', 'MarkerFaceColor', 'k');
    title(['1/(1+x^2), n=' num2str(n)]);
    grid on;
end

% Fonction pour calculer les coefficients ( Newton )
function coeffs = newton_coeffs(x, y)
    n = length(x);
    coeffs = y;
    for j = 2:n
        for i = n:-1:j
            coeffs(i) = (coeffs(i) - coeffs(i-1)) / (x(i) - x(i-j+1));
        end
    end
end

% Fonction pour évaluer le polynome ( Newton )
function y_interp = newton_eval(coeffs, x_nodes, x_eval)
    n = length(x_nodes);
    y_interp = coeffs(n) * ones(size(x_eval));
    for k = n-1:-1:1
        y_interp = y_interp .* (x_eval - x_nodes(k)) + coeffs(k);
    end
end
```

Listing 1 : Interpolation par différences divisées de Newton (points équidistants)

Programme MATLAB de tracé des courbes de la fonction interpolée et du polynôme interpolateur en utilisant l'algorythme de Newton avec la méthode des points de Tchebychev.

```
% Interpolation par la méthode des différences divisées de Newton (
    Points de Tchebychev)
clc;
clear;
close all;
```

```

% Définition des fonctions f1 et f2
f1 = @(x) cos(x);
f2 = @(x) 1 ./ (1 + x.^2);

% Bornes de l'intervalle
a = -5;
b = 5;

% Choix du nombre de points pour l'interpolation
n_values = [3, 5, 11];

% Evaluation et tracé pour chaque valeur de n
x_fine = linspace(a, b, 1000);
figure;

for idx = 1:length(n_values)
    n = n_values(idx);

    % --- Points de Tchebychev ---
    % Formule pour l'intervalle [a, b]
    k = 0:n-1;
    x_n = 0.5*(a + b) + 0.5*(b - a) * cos((2*k + 1) * pi / (2*n));

    y_f1 = f1(x_n);
    y_f2 = f2(x_n);

    % Calcul des coefficients de Newton
    coeffs_f1 = newton_coeffs(x_n, y_f1);
    coeffs_f2 = newton_coeffs(x_n, y_f2);

    % Evaluation des polynomes interpolants
    y_interp_f1 = newton_eval(coeffs_f1, x_n, x_fine);
    y_interp_f2 = newton_eval(coeffs_f2, x_n, x_fine);

    % Tracé des résultats
    subplot(2, length(n_values), idx);
    plot(x_fine, f1(x_fine), 'b-', 'LineWidth', 1.5); hold on;
    plot(x_fine, y_interp_f1, 'r--', 'LineWidth', 1.5);
    plot(x_n, y_f1, 'ko', 'MarkerFaceColor', 'k');
    title(['cos(x), Tchebychev n=' num2str(n)]);
    legend('cos(x)', 'Interp.', 'Points', 'Location', 'Best');
    grid on;

    subplot(2, length(n_values), idx + length(n_values));
    plot(x_fine, f2(x_fine), 'b-', 'LineWidth', 1.5); hold on;
    plot(x_fine, y_interp_f2, 'r--', 'LineWidth', 1.5);
    plot(x_n, y_f2, 'ko', 'MarkerFaceColor', 'k');
```

```
    title(['1/(1+x^2), Tchebychev n=' num2str(n)]);
    legend('1/(1+x^2)', 'Interp.', 'Points', 'Location', 'Best');
    grid on;
end

% Fonction pour calculer les coefficients ( Newton )
function coeffs = newton_coeffs(x, y)
    n = length(x);
    coeffs = y;
    for j = 2:n
        for i = n:-1:j
            coeffs(i) = (coeffs(i) - coeffs(i-1)) / (x(i) - x(i-j+1));
        end
    end
end

% Fonction pour evaluer le polynome ( Newton )
function y_interp = newton_eval(coeffs, x_nodes, x_eval)
    n = length(x_nodes);
    y_interp = coeffs(n) * ones(size(x_eval));
    for k = n-1:-1:1
        y_interp = y_interp .* (x_eval - x_nodes(k)) + coeffs(k);
    end
end
```

Listing 2 : Interpolation de Newton optimisée par les points de Tchebychev